

Методичка 13. ООП: классы, объекты, методы, инкапсуляция

1. Зачем нужна эта тема

ООП нужно, когда программа начинает работать не просто с отдельными числами, строками и списками, а с полноценными сущностями.

Например, в программе могут быть:

- банковская карта;
- пользователь;
- ученик;
- питомец;
- товар;
- персонаж игры;
- заказ;
- фильм;
- книга.

У каждой сущности есть данные и действия.

Например, у банковской карты могут быть данные:

- номер карты;
- имя владельца;
- баланс;
- срок действия.

И действия:

- пополнить баланс;
- снять деньги;
- проверить баланс;
- заблокировать карту.

До ООП данные и функции часто живут отдельно. Например, баланс хранится в переменной, а операция пополнения — в отдельной функции. В ООП данные и

действия объединяются внутри объекта.

Это делает код понятнее: объект сам отвечает за своё состояние и за действия, которые с ним можно выполнять.

Эта тема продолжает тему `dataclasses`, но делает следующий шаг. `dataclass` в основном хранит данные, а обычный класс может хранить данные и управлять поведением объекта.

2. Главная идея темы

ООП — это способ писать программы через объекты.

Объект — это “вещь” внутри программы. У объекта есть:

- поля — данные объекта;
- методы — действия объекта.

Например:

```
card.add_money(1000)
card.withdraw_money(500)
card.print_info()
```

Такой код читается почти как обычный текст:

- пополнить карту на 1000;
- снять с карты 500;
- вывести информацию о карте.

ООП — объектно-ориентированное программирование.

Класс — чертёж объекта. Он описывает, какие данные и методы будут у объекта.

Объект — конкретная сущность, созданная по классу.

Поле / атрибут — данные объекта.

Метод — функция внутри класса.

`self` — ссылка на текущий объект.

`__init__` — специальный метод, который вызывается при создании объекта.

Инкапсуляция — идея, что объект сам управляет своими данными и не позволяет ломать себя неправильными действиями.

3. Базовый синтаксис

Создание класса

```
class BankCard:
    pass
```

Так создаётся пустой класс `BankCard` .

Пока в нём нет данных и методов.

Создание объекта

```
class BankCard:
    pass

card = BankCard()

print(card)
```

`card` — это объект класса `BankCard` .

Класс — это чертёж. Объект — конкретная созданная карта.

Метод `__init__`

```
class BankCard:
    def __init__(self, owner, balance):
        self.owner = owner
        self.balance = balance

card = BankCard("Иван", 1000)

print(card.owner)
print(card.balance)
```

Что происходит:

- `__init__` вызывается при создании объекта;
- `owner` и `balance` передаются при создании;
- `self.owner` и `self.balance` сохраняются внутри объекта.

Метод объекта

```
class BankCard:
    def __init__(self, owner, balance):
        self.owner = owner
        self.balance = balance

    def print_info(self):
        print("Владелец:", self.owner)
        print("Баланс:", self.balance)

card = BankCard("Иван", 1000)

card.print_info()
```

`print_info()` — это метод. Он вызывается у конкретного объекта.

Метод с параметром

```
class BankCard:
    def __init__(self, owner, balance):
        self.owner = owner
        self.balance = balance

    def add_money(self, amount):
        self.balance = self.balance + amount

card = BankCard("Иван", 1000)

card.add_money(500)

print(card.balance)
```

Метод `add_money()` принимает сумму и увеличивает баланс.

4. Разбор темы от простого к сложному

Шаг 1. Класс как описание сущности

```
class Cat:
    def __init__(self, name, age):
        self.name = name
        self.age = age
```

Класс `Cat` описывает кошку.

У каждой кошки есть:

- имя;
- возраст.

Шаг 2. Объекты одного класса

```
class Cat:
    def __init__(self, name, age):
        self.name = name
        self.age = age

cat1 = Cat("Барсик", 3)
cat2 = Cat("Мурка", 5)

print(cat1.name)
print(cat2.name)
```

Один класс может создать много объектов.

`cat1` и `cat2` — разные объекты. У каждого своё имя и свой возраст.

Шаг 3. Метод как действие объекта

```
class Cat:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def say_meow(self):
        print(self.name, "говорит: мяу")

cat = Cat("Барсик", 3)

cat.say_meow()
```

Метод `say_meow()` относится к конкретному коту.

Шаг 4. Изменение состояния объекта

```
class Cat:
    def __init__(self, name, energy):
        self.name = name
        self.energy = energy

    def sleep(self):
        self.energy = self.energy + 10

cat = Cat("Барсик", 50)

cat.sleep()

print(cat.energy)
```

У объекта есть состояние. Методы могут это состояние менять.

Шаг 5. Проверка внутри метода

```
class BankCard:
    def __init__(self, owner, balance):
        self.owner = owner
        self.balance = balance

    def withdraw_money(self, amount):
        if amount <= self.balance:
            self.balance = self.balance - amount
            print("Деньги сняты")
        else:
            print("Недостаточно денег")

card = BankCard("Иван", 1000)

card.withdraw_money(300)
card.withdraw_money(2000)
```

Объект сам проверяет, можно ли снять деньги.

Это лучше, чем менять баланс напрямую снаружи.

Шаг 6. Инкапсуляция

Плохой вариант:

```
card.balance = -100000
```

Так можно случайно сломать объект. У карты не должно быть отрицательного баланса, если программа этого не разрешает.

Более аккуратный вариант:

```
class BankCard:
    def __init__(self, owner, balance):
        self.owner = owner
        self.__balance = balance

    def add_money(self, amount):
        if amount > 0:
            self.__balance = self.__balance + amount

    def withdraw_money(self, amount):
        if 0 < amount <= self.__balance:
            self.__balance = self.__balance - amount
            print("Деньги сняты")
        else:
            print("Операция невозможна")

    def get_balance(self):
        return self.__balance
```

`__balance` показывает, что баланс не нужно менять напрямую снаружи.

Доступ к балансу идёт через методы:

```
card = BankCard("Иван", 1000)

card.add_money(500)
card.withdraw_money(300)

print(card.get_balance())
```

Шаг 7. Хранилище объектов

```
class CardsStorage:
    def __init__(self):
        self.cards = []

    def add_card(self, card):
        self.cards.append(card)

    def print_cards(self):
        for card in self.cards:
            print(card.owner, card.get_balance())
```

Один объект может хранить другие объекты.

Например, `CardsStorage` хранит список банковских карт.

Шаг 8. Предметные методы вместо get/set-стиля

Не очень хороший стиль:

```
card.set_balance(card.get_balance() - 500)
```

Лучше:

```
card.withdraw_money(500)
```

Почему второй вариант лучше:

- он читается понятнее;
- внутри метода можно сделать проверку;
- объект сам защищает свои данные.

5. Частые ошибки учеников

Ошибка 1. Забывают `self`

Неправильно:

```
class Cat:
    def __init__(name, age):
        self.name = name
        self.age = age
```

Почему это ошибка:

Первым параметром метода должен быть `self`. Через него объект обращается к своим данным.

Правильно:

```
class Cat:
    def __init__(self, name, age):
        self.name = name
        self.age = age
```

Ошибка 2. Путают класс и объект

Неправильно:

```
class Cat:
    def __init__(self, name):
        self.name = name

print(Cat.name)
```

Почему это ошибка:

`Cat` — это класс, чертёж. Имя есть у конкретного объекта, а не у класса в целом.

Правильно:

```
cat = Cat("Барсик")

print(cat.name)
```

Ошибка 3. Забывают вызвать метод

Неправильно:

```
cat.say_meow
```

Почему это ошибка:

Без круглых скобок метод не вызывается.

Правильно:

```
cat.say_meow()
```

Ошибка 4. Создают поле без `self`

Неправильно:

```
class Cat:
    def __init__(self, name):
        name = name
```

Почему это ошибка:

Так значение не сохраняется внутри объекта.

Правильно:

```
class Cat:
    def __init__(self, name):
        self.name = name
```

Ошибка 5. Меняют внутренние данные напрямую

Неправильно:

```
card.balance = -5000
```

Почему это плохо:

Так можно создать неправильное состояние объекта.

Правильно:

```
card.withdraw_money(5000)
```

Метод сам проверит, можно ли выполнить операцию.

Ошибка 6. Делают класс без поведения

Неправильно:

```
class BankCard:
    def __init__(self, owner, balance):
        self.owner = owner
        self.balance = balance
```

Почему это не всегда хорошо:

Класс только хранит данные, но не управляет ими. Для простого хранения можно использовать `dataclass`.

Лучше добавить методы:

```
class BankCard:
    def __init__(self, owner, balance):
        self.owner = owner
        self.__balance = balance

    def add_money(self, amount):
        if amount > 0:
            self.__balance = self.__balance + amount

    def get_balance(self):
        return self.__balance
```

Ошибка 7. Пишут слишком много setter/getter без смысла

Неправильно:

```
card.set_balance(1000)
card.set_balance(-500)
```

Почему это плохо:

Если любой код может поставить любой баланс, объект не защищён.

Лучше:

```
card.add_money(1000)
card.withdraw_money(500)
```

Методы должны отражать реальные действия.

6. Мини-примеры для закрепления

Пример 1. Класс `Student`

```
class Student:
    def __init__(self, name, grade):
        self.name = name
        self.grade = grade

student = Student("Аня", 9)

print(student.name)
print(student.grade)
```

Программа создаёт объект ученика. Закрепляется класс, объект и `__init__`.

Пример 2. Метод вывода информации

```
class Student:
    def __init__(self, name, grade):
        self.name = name
        self.grade = grade

    def print_info(self):
        print(self.name, "учится в", self.grade, "классе")

student = Student("Аня", 9)

student.print_info()
```

Закрепляется метод объекта.

Пример 3. Класс `Counter`

```
class Counter:
    def __init__(self):
        self.value = 0

    def increase(self):
        self.value = self.value + 1

counter = Counter()

counter.increase()
counter.increase()

print(counter.value)
```

Закрепляется изменение состояния объекта.

Пример 4. Проверка внутри метода

```
class Wallet:
    def __init__(self, money):
        self.money = money

    def spend(self, amount):
        if amount <= self.money:
            self.money = self.money - amount
            print("Покупка выполнена")
        else:
            print("Недостаточно денег")

wallet = Wallet(1000)

wallet.spend(300)
wallet.spend(2000)
```

Закрепляется защита состояния через метод.

Пример 5. Приватное поле

```
class Wallet:
    def __init__(self, money):
        self.__money = money

    def get_money(self):
        return self.__money

wallet = Wallet(1000)

print(wallet.get_money())
```

Закрепляется идея инкапсуляции.

Пример 6. Список объектов

```
class Book:
    def __init__(self, title):
        self.title = title

books = [
    Book("Книга 1"),
    Book("Книга 2"),
    Book("Книга 3")
]

for book in books:
    print(book.title)
```

Закрепляется хранение объектов в списке.

Пример 7. Хранилище объектов

```
class Storage:
    def __init__(self):
        self.items = []

    def add_item(self, item):
        self.items.append(item)

    def print_items(self):
        for item in self.items:
            print(item)

storage = Storage()

storage.add_item("яблоко")
storage.add_item("банан")

storage.print_items()
```

Закрепляется объект, который управляет списком.

7. Практические задания

Лёгкий уровень

1. Создать класс `Student` с полями `name` и `age`.
2. Создать объект `Student` и вывести его имя.

3. Создать класс `Book` с полями `title` и `author`.
4. Добавить в класс `Book` метод `print_info()`.
5. Создать класс `Counter`, у которого есть поле `value` и метод `increase()`.

Средний уровень

1. Создать класс `Wallet` с балансом и методами `add_money()` и `spend_money()`.
2. Создать класс `Player` с полями `name`, `health` и методом `take_damage()`.
3. Создать класс `Product` с полями `title`, `price`, `count` и методом `print_info()`.
4. Создать список объектов `Book` и вывести названия всех книг.
5. Создать класс `Storage`, который хранит список товаров и умеет добавлять товар.

Повышенный уровень

1. Создать класс `BankCard`, который не позволяет сделать баланс отрицательным.
2. Создать класс `Pet` с полями `name`, `hunger`, `energy` и методами `feed()`, `sleep()`, `play()`.
3. Создать хранилище объектов, которое умеет добавлять объект, искать объект по названию и выводить все объекты.

8. Контрольные вопросы

1. Что такое ООП?
2. Что такое класс?
3. Что такое объект?
4. Чем класс отличается от объекта?
5. Для чего нужен `__init__`?
6. Что такое `self`?
7. Что такое метод?
8. Чем метод отличается от обычной функции?
9. Что такое состояние объекта?
10. Что такое инкапсуляция?
11. Почему не всегда хорошо менять поля напрямую?
12. Почему методы лучше называть как реальные действия?

9. Краткий конспект в конце

Главная идея темы: ООП позволяет объединять данные и действия внутри объектов.

Основная конструкция:

```
class Cat:
    def __init__(self, name):
        self.name = name

    def say_meow(self):
        print(self.name, "говорит мяу")
```

Создание объекта:

```
cat = Cat("Барсик")
```

Вызов метода:

```
cat.say_meow()
```

Инкапсуляция:

```
class Wallet:
    def __init__(self, money):
        self.__money = money

    def add_money(self, amount):
        if amount > 0:
            self.__money = self.__money + amount
```

Важно запомнить:

- класс — чертёж;
- объект — конкретная сущность;
- `__init__` вызывается при создании объекта;
- `self` указывает на текущий объект;
- методы описывают действия объекта;
- инкапсуляция защищает объект от неправильного состояния;
- лучше писать методы предметной области: `add_money`, `withdraw_money`, `feed`, `attack`.

10. Что ученик должен уметь после методички

После этой темы ученик должен уметь:

- создавать простые классы;
- создавать объекты;
- использовать `__init__`;
- понимать роль `self`;
- добавлять методы в класс;
- изменять состояние объекта через методы;
- использовать приватные поля на базовом уровне;
- объяснять смысл инкапсуляции;
- хранить объекты в списке;
- писать простые классы для учебных задач.